

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

Про виконання лабораторної роботи №3
«Створення класів»

Виконала:

Студентка групи ФЕП-11

Матла Діана Ярославівна

Перевірив:

Асистент Левуш Павло Назарович

Львів 2026

Мета: Дослідити та реалізувати чисельні методи знаходження коренів нелінійних рівнянь, зокрема метод дихотомії (бісекції) та метод Ньютона. Навчитися застосовувати ці методи для заданої функції, оцінювати їх збіжність, точність та обчислювальну ефективність, а також порівняти результати, отримані різними підходами.

Хід роботи

Файли та готовий код було відправлено на github за посиланням:

<https://github.com/Diana-Matla/OOP/tree/main/lab1>

- Було створено клас `Dyhotomia_class`, який містить:
 1. змінні для меж відрізка a , b та точності ϵ
 2. функцію $f(x)f(x)f(x)$
 3. методи знаходження кореня: **дихотомія** та **Ньютон**

- Реалізовано метод дихотомії:
 1. перевірка на наявність кореня на відрізку через зміну знаків функції
 2. поступове ділення відрізка навпіл
 3. уточнення інтервалу, в якому знаходиться корінь
 4. завершення при досягненні заданої точності ϵ

- Реалізовано метод Ньютона:
 1. вибір початкового наближення як середини відрізка
 2. обчислення похідної функції
 3. ітераційне уточнення кореня за формулою Ньютона
 4. зупинка при досягненні точності або збіжності

```

C Dyhotomia_class.h > Dyhotomia_class > eps
1  #ifndef DYHOTOMIA_CLASS_H
2  #define DYHOTOMIA_CLASS_H
3
4  class Dyhotomia_class
5  {
6  private:
7      double a;
8      double b;
9      double eps;
10     double df(double x);
11
12 public:
13     Dyhotomia_class(void);
14     ~Dyhotomia_class(void);
15
16     void setVolumes (double vol_a, double vol_b);
17     void setTolerance (double vol_eps);
18     double f(double x);
19     double Dyhotomia();
20     double Newton();
21 };
22
23 void result();
24
25 #endif

```

Dyhotomia.h

```

G main.cpp > main()
1  #include "Dyhotomia_class.cpp"
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      result();
7      return 0;
8  }

```

Main.cpp

```

Dyhotomia_class.cpp > Dyhotomia_class()
1  #include "Dyhotomia_Class.h"
2  #include<iostream>
3  #include <cmath>
4  using namespace std;
5
6  Dyhotomia_class::Dyhotomia_class() {
7      a = 0;
8      b = 0;
9      eps = 0;
10 }
11
12 Dyhotomia_class::~Dyhotomia_class() {}
13
14 void Dyhotomia_class::setVolumes (double vol_a, double vol_b){
15     a = vol_a;
16     b = vol_b;
17 }
18
19 void Dyhotomia_class::setTolerance (double vol_eps){
20     eps = vol_eps;
21 }
22
23
24 double Dyhotomia_class::f(double x) {
25     if (fabs(x) < 1e-12)
26         return NAN;
27     return cos(2/x) - 2*sin(1/x) + 1/x;
28 }

```

```

double Dyhotomia_class::df(double x){
    double h = 1e-6;
    return (f(x + h) - f(x - h)) / (2*h);
}

double Dyhotomia_class::Dyhotomia() {
    double left = a;
    double f_left = f(left);
    double right = b;
    double f_right = f(right);

    if (isnan(f_left) || isnan(f_right)) {
        cout << "Function undefined in interval" << endl;
        return NAN;
    }

    if (f_left * f_right > 0) {
        cout << "No root in interval" << endl;
        return NAN;
    }

    if (fabs(f_left) < eps)
        return left;

    if (fabs(f_right) < eps)
        return right;
}

```

```

double mid;
double f_mid;

//for (int i = 0; i < 1000; i++) {
while (fabs(right - left) >= eps) {
    mid = (left + right) / 2.0;
    f_mid = f(mid);

    if (fabs(f_mid) < eps)
        return mid;

    if (f_left * f_mid < 0) {
        right = mid;
        f_right = f_mid;
    } else {
        left = mid;
        f_left = f_mid;
    }
}

return (left + right) / 2;
}

```

```

double Dyhotomia_class::Newton(){
    double x = (a + b) / 2;
    if (fabs(f(x)) < eps)
        return x;

    double dfx;
    double x_new;

    for (int i = 0; i < 100; i++) {
        dfx = df(x);

        if (fabs(dfx) < 1e-12) {
            cout << "Derivative too small" << endl;
            return NAN;
        }

        x_new = x - f(x) / dfx;
        if (fabs(f(x_new)) < eps)
            return x_new;

        if (fabs(x_new - x) < eps)
            return x_new;
    }
}

```

```

for (int i = 0; i < 100; i++) {
    dfx = df(x);

    if (fabs(dfx) < 1e-12) {
        cout << "Derivative too small" << endl;
        return NAN;
    }

    x_new = x - f(x) / dfx;
    if (fabs(f(x_new)) < eps)
        return x_new;

    if (fabs(x_new - x) < eps)
        return x_new;

    x = x_new;
}

cout << "Newton did not converge" << endl;
return x;
}

```

```

void result(){
    Dyhotomia_class* dyh = new Dyhotomia_class();

    dyh->setVolumes(1, 2);
    dyh->setTolerance(0.0001);

    cout << "Dyhotomia: " << dyh->Dyhotomia() << endl;
    cout << "Newton: " << dyh->Newton() << endl;

    delete dyh;
}

```

Вивід з терміналу

```

PS D:\labs\OOP\lab 3> cd "d:\
Dyhotomia: 1.87549
Newton: 1.87561
PS D:\labs\OOP\lab 3>

```

Висновок: У ході роботи було реалізовано та досліджено два чисельні методи знаходження коренів нелінійного рівняння — метод дихотомії та метод Ньютона. Метод дихотомії забезпечує гарантовану збіжність за умови зміни знаку функції на заданому інтервалі, проте має відносно повільну швидкість обчислень. Метод

Ньютона є значно швидшим, однак його збіжність залежить від вибору початкового наближення та поведінки функції.